

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Operating Systems

COURSE CODE: CSA0403

COURSE OUTCOME COVERED

CO5: Analyze the components and functionalities of Linux operating systems and virtualization environments to enhance their operational efficiency

Assessment Tool

Weightage: 35%

QUESTIONS: 5

Total Marks:25

Annexure A

CONSTRAINT BASED PROBLEM SOLVING

Code of Conduct:

I, **Ashira Bridget Susan S (192524316)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



ANNEXURE A

CONSTRAINT BASED PROBLEM SOLVING

S.No. 1

Question:

A server with a total storage capacity of 50 GB needs to effectively manage space for rapidly growing system logs, user files, and application data. Analyze the scenario and design an optimized Linux file system layout, incorporating suitable partitioning techniques and storage management tools (such as log rotation), to ensure efficient utilization of disk space and maintain system stability. The solution must ensure that log files do not exceed 10 GB and that critical services continue to operate without interruption due to disk space exhaustion.

Answer:

For a 50 GB Linux server, separate partitions should be created for system files, logs, user data, temporary files, and swap space. This prevents one type of data from consuming the entire disk.

Suggested File System Layout:

Partition	Size	Purpose
/	8 GB	Operating system and system files
/var	7 GB	Application and variable data
/var/log	8 GB	System and application logs
/home	15 GB	User files
/opt	5 GB	Application software
/tmp	2 GB	Temporary files
swap	4 GB	Virtual memory
Free/Reserved	1 GB	Emergency space
Total	50 GB	

The /var/log directory is placed on a separate partition of 8 GB. Therefore, log files cannot consume more than this partition. This is safer than allowing logs to share the root partition.

Log Management:

Linux logrotate can be configured to automatically rotate logs based on size or time. For example:

```
sudo logrotate /etc/logrotate.conf
```

A log rotation policy can be configured to:

- Rotate logs daily or when they reach a specified size.
- Compress old log files.
- Keep only a fixed number of old log files.
- Delete very old logs automatically.

For systems using systemd-journald, a limit can also be configured using:

```
SystemMaxUse=7G
```

This ensures that journald does not consume the entire log partition.

Additional Storage Management:

Commands such as the following can be used to monitor disk usage:

```
df -h
```

```
du -sh /var/log/*
```

A monitoring script can generate alerts when disk usage reaches, for example, 80% or 90%.

Conclusion:

Separating /var/log, /home, and other major directories prevents user files or log files from exhausting the root partition. Log rotation, compression, monitoring, and reserved free space ensure that critical services continue to operate without interruption. The log partition is limited to 8 GB, which is safely below the required 10 GB maximum.

S.No. 2

Question:

A Linux system handling high CPU workloads suffers from frequent context switching delays. Given that hardware upgrades are not possible and real-time processing must be supported, evaluate kernel-level optimization techniques such as scheduler tuning and the use of kernel modules—to improve overall system performance.

Answer:

Frequent context switching occurs when the CPU spends too much time switching between processes instead of executing useful work. For real-time processing, Linux can be optimized using scheduler configuration, CPU affinity, process priorities, and an appropriate kernel.

1. Use a Real-Time Kernel:

For applications requiring predictable response times, a Linux kernel with the PREEMPT_RT real-time patch can be used. It reduces scheduling latency and improves deterministic behavior.

2. Set Process Priority:

Normal processes can be assigned different priorities using nice and renice.

```
nice -n -10 ./program
```

An existing process can be adjusted using:

```
sudo renice -10 -p PID
```

For real-time scheduling, chrt can be used:

```
sudo chrt -f 50 ./program
```

3. Use CPU Affinity:

Processes can be restricted to specific CPU cores using taskset.

```
taskset -c 2 ./program
```

This reduces unnecessary movement of processes between CPU cores and can improve cache utilization.

4. CPU Isolation:

For demanding real-time applications, selected CPU cores can be isolated from normal workloads using kernel boot parameters such as isolcpus and, where appropriate, nohz_full.

This allows dedicated CPU cores to handle critical real-time tasks.

5. Kernel Modules:

Kernel modules should be used only when required for specific hardware or kernel functionality. Unnecessary modules and drivers can be disabled or unloaded to reduce system overhead.

Modules can be viewed using:

```
lsmod
```

6. Disable Unnecessary Services:

Background services that consume CPU resources should be identified and stopped if they are not required.

```
systemctl --type=service
```

Conclusion:

The best approach is to use a real-time/preemptible kernel, assign suitable process priorities, use CPU affinity, isolate CPU cores for critical tasks, and remove unnecessary background activity. These techniques reduce scheduling overhead and context-switching delays without requiring hardware upgrades.

S.No. 3

Question:

An administrator must process 1 million log entries per day using only shell scripting. Analyze the problem and design an efficient solution using optimized Linux commands that ensures the script completes within 2 minutes. Justify the choice of commands used for achieving high performance.

Answer:

Processing 1 million log entries requires minimizing unnecessary commands, avoiding repeated loops, and using optimized Linux utilities that process large amounts of text efficiently.

Instead of processing each line individually using a shell loop, commands such as awk, grep, sort, and cut should be used because they process data in batches.

Example Requirement: Count the number of log entries containing the word ERROR.

```
#!/bin/bash
```

```
LOG="/var/log/application.log"
```

```
grep -c "ERROR" "$LOG"
```

For more complex processing, awk can be used:

```
awk '/ERROR/ {count++} END {print count}' "$LOG"
```

For example, to count errors by date:

```
awk '$3 == "ERROR" {count[$1]++} END {for (d in count) print d, count[d]}' "$LOG"
```

To extract frequently occurring error messages:

```
grep "ERROR" "$LOG" | sort | uniq -c | sort -nr
```

For very large files, unnecessary commands should be avoided. For example, instead of:

```
cat file.log | grep ERROR
```

use:

```
grep ERROR file.log
```

This avoids creating an unnecessary pipeline process.

Performance Optimization Techniques:

1. Use awk for filtering and calculations in a single operation.
2. Use grep for fast pattern matching.
3. Avoid line-by-line shell while loops wherever possible.
4. Avoid unnecessary cat, sed, and multiple pipelines.
5. Process only the required fields instead of the entire log.
6. Use parallel processing only when the task can safely be divided.
7. Measure execution time using:

```
time ./process_logs.sh
```

Example Optimized Script:

```
#!/bin/bash
```

```
LOG="/var/log/application.log"
```

```
awk '  
/ERROR/ { errors++ }  
/WARNING/ { warnings++ }  
END {  
    print "Errors:", errors  
    print "Warnings:", warnings  
}  
' "$LOG"
```

awk reads the file once and performs multiple operations during the same pass.

Conclusion:

For 1 million log entries, batch-oriented commands such as awk and grep are more efficient than shell loops. Minimizing process creation, reducing the number of passes over the file, and processing only required fields can allow the script to meet the two-minute requirement, subject to the server's CPU, storage, and log format.

S.No. 4

Question:

A Linux system runs multiple background services, where a faulty process is consuming 90% of CPU resources. Without restarting the system and while ensuring critical services remain unaffected, analyze the situation and propose a process management strategy using appropriate Linux tools to identify, monitor, and control the faulty process.

Answer:

A process consuming 90% CPU should first be identified and monitored before taking corrective action. The process should not be terminated immediately because it may belong to a critical service.

1. Identify the Faulty Process:

Use top to view CPU usage:

top

Alternatively:

```
ps aux --sort=-%cpu | head
```

This displays the processes consuming the highest CPU resources.

2. Identify the Process Details:

Once the PID is identified:

```
ps -p PID -o pid,ppid,user,stat,%cpu,%mem,cmd
```

This helps determine the process owner, parent process, CPU usage, and command.

3. Monitor the Process:

The process can be monitored using:

```
top -p PID
```

or:

```
pidstat -p PID 1
```

This helps determine whether the high CPU usage is temporary or continuous.

4. Reduce CPU Priority:

If the process is not critical but must continue running, its priority can be reduced:

```
sudo renice 10 -p PID
```


This allows other important processes to receive more CPU time.

5. Restrict CPU Usage:

If necessary, systemd resource controls or tools such as cputime can be used to limit CPU consumption.

For a systemd service, CPU limits can be configured using appropriate service resource controls.

6. Stop or Terminate the Process:

If the process is confirmed to be faulty and cannot be controlled:

kill PID

If it does not respond:

kill -9 PID

SIGKILL should be used only as a last resort because the process cannot perform cleanup operations.

7. Protect Critical Services:

Before changing or terminating a process, check whether it belongs to a critical service:

systemctl status service-name

Critical services should be left untouched.

Conclusion:

The recommended strategy is to identify the process using top or ps, verify its importance, monitor it using pidstat, reduce its priority or restrict its CPU usage, and terminate it only if necessary. This approach controls the faulty process without restarting the server or unnecessarily affecting critical services.

S.No. 5

Question:

A physical server must host 5 virtual machines with a total available RAM of 16 GB. Each VM requires a minimum of 2 GB, while the host operating system needs 4 GB. Analyze the scenario and propose an efficient resource allocation strategy using virtualization concepts to ensure all virtual machines operate smoothly under the given constraints.

Answer:

The server has a total of **16 GB RAM**.

Memory required by the host operating system:

Host RAM = 4 GB

Memory required by the five virtual machines:

$5 \times 2 \text{ GB} = 10 \text{ GB}$

Therefore:

Total required RAM = 4 GB + 10 GB = 14 GB

Remaining memory:

$16 \text{ GB} - 14 \text{ GB} = 2 \text{ GB}$

Thus, the requirements can be satisfied, with **2 GB remaining as a reserve**.

Proposed Allocation:

Resource	RAM Allocation
Host OS	4 GB
VM 1	2 GB
VM 2	2 GB
VM 3	2 GB
VM 4	2 GB
VM 5	2 GB

Resource	RAM Allocation
Reserved/Overhead	2 GB
Total	16 GB

Each VM receives its required minimum of 2 GB, while the host receives 4 GB.

Virtualization Techniques:

1. **Memory Reservation:**

Reserve at least 4 GB for the host so that the host OS can continue operating reliably.

2. **Dynamic Memory Allocation:**

If supported by the hypervisor, memory can be dynamically allocated between VMs depending on workload.

3. **Memory Ballooning:**

Ballooning can reclaim unused memory from less active VMs and provide it to VMs experiencing higher demand.

4. **Avoid Overcommitment:**

Since only 2 GB is available beyond the minimum requirements, excessive memory overcommitment should be avoided.

5. **Prioritize Important VMs:**

Critical VMs can be given higher resource priority, while less important VMs can use their minimum allocation when necessary.

6. **Monitor Memory Usage:**

On the Linux host, memory can be monitored using:

`free -h`

and:

`top`

The hypervisor's monitoring tools should also be used to observe individual VM memory consumption.

Conclusion:

The allocation of **4 GB to the host and 2 GB to each of the five VMs** requires 14 GB, leaving **2 GB as reserve memory**. This satisfies the minimum requirements of all VMs while maintaining sufficient memory for the host. Dynamic allocation and ballooning can be used when supported, but excessive memory overcommitment should be avoided to ensure smooth operation.

S.No. 6

Question:

A Linux server has 16 GB RAM and runs several applications simultaneously. During peak hours, the system begins using swap heavily even though some applications appear idle. Analyze the memory-management behavior and recommend Linux commands and configuration techniques to identify the cause and optimize memory utilization.

Answer:

Heavy swap usage does not always mean that the system has completely run out of RAM. Linux may move inactive memory pages from RAM to swap so that RAM can be used more efficiently for active applications and file caching.

During peak hours, applications that appear idle may still have memory allocated to them. The kernel may move their less frequently used pages to swap.

1. Check RAM and Swap Usage:

The free command provides an overview of memory usage:

```
free -h
```

The top command can be used to identify processes consuming large amounts of memory:

```
top
```

Processes can also be sorted by memory usage using:

```
ps aux --sort=-%mem | head
```

2. Check Swap Activity:

The vmstat command can be used to monitor memory and swap activity:

```
vmstat 1
```

The si and so columns indicate swap-in and swap-out activity.

The currently configured swap areas can be viewed using:

```
swapon --show
```

3. Identify Memory-Hungry Applications:

The administrator should identify applications with unusually high memory consumption and determine whether they have memory leaks or unnecessarily large memory allocations.

For a specific process:

pmap PID

This displays the memory mapping of the process.

4. Adjust Swappiness:

Linux uses the vm.swappiness parameter to control how aggressively it uses swap.

The current value can be checked using:

```
cat /proc/sys/vm/swappiness
```

A lower value can reduce unnecessary swapping:

```
sudo sysctl vm.swappiness=10
```

To make the setting persistent, add the following to /etc/sysctl.conf:

```
vm.swappiness=10
```

The value should be chosen based on workload rather than blindly setting it to the minimum.

5. Check for Memory Leaks:

If a process continuously increases its memory consumption, it may have a memory leak. Monitoring the process over time using top, ps, or application-specific monitoring tools can help identify this problem.

6. Optimize Applications:

Unnecessary applications and services should be stopped where appropriate.

Applications should also be configured to use reasonable memory limits.

7. Ensure Adequate Swap Space:

Swap should not be considered a replacement for physical RAM, but an appropriately sized swap area can prevent abrupt failures when memory demand temporarily increases.

Conclusion:

The administrator should first determine whether swap usage is caused by actual memory pressure or normal kernel behavior. Commands such as free, top, vmstat, ps, and swapon can identify the cause. Adjusting vm.swappiness, controlling memory-heavy applications, fixing memory leaks, and maintaining suitable swap space can improve memory utilization and reduce excessive swapping.

S.No. 7

Question:

A Linux system contains thousands of files distributed across multiple directories. A backup process is taking several hours to complete. Analyze the directory structure and design an efficient command-line strategy using suitable Linux utilities to identify large files, duplicate files, and unnecessary temporary data.

Answer:

A backup process may become slow because of a large number of files, very large files, duplicate data, and unnecessary temporary files. The first step is to analyze disk usage and identify files that do not need to be included in the backup.

1. Analyze Directory Usage:

The du command can identify which directories consume the most space:

```
du -h --max-depth=1 /home
```

For the entire filesystem:

```
sudo du -xh --max-depth=1 /
```

2. Find Large Files:

Files larger than 1 GB can be identified using:

```
find /home -type f -size +1G -ls
```

For example, files larger than 500 MB:

```
find /home -type f -size +500M -ls
```

These files can be reviewed to determine whether they are necessary.

3. Identify Duplicate Files:

Duplicate files waste storage and increase backup time. A checksum can be calculated using:

```
md5sum file1 file2
```

For larger collections, tools such as fdupes can be used:

```
fdupes -r /home
```

Duplicate files should be removed only after verifying that they are safe to delete.

4. Identify Temporary Files:

Temporary files can be located using:

```
find /tmp -type f -mtime +7
```

This identifies files in /tmp that have not been modified for more than seven days.

Similar searches can be performed for application-specific temporary directories.

5. Find Old Files:

Files that have not been accessed or modified for a long period can be identified:

```
find /home -type f -mtime +90
```

These files can be reviewed for archiving or removal.

6. Improve the Backup Strategy:

The backup process should exclude unnecessary directories such as caches and temporary files.

For example, with rsync:

```
rsync -av --exclude='/tmp/' --exclude='*.cache' /home/ /backup/home/
```

rsync is efficient because it can copy only changed files during subsequent backups.

7. Compress Suitable Data:

Old log files and archived data can be compressed before backup:

```
gzip old_log.txt
```

This reduces the amount of data that must be transferred.

Conclusion:

The administrator should first analyze directory usage using du, locate large files using find, identify duplicates using checksums or fdupes, and remove or exclude unnecessary temporary files. Using rsync for incremental backups can significantly reduce backup time because unchanged files do not need to be transferred repeatedly.

S.No. 8

Question:

A Linux server has four CPU cores and supports multiple users running computationally intensive programs. Analyze how Linux scheduling manages these processes and propose a strategy using process priorities and CPU-affinity techniques to improve responsiveness for critical applications.

Answer:

Linux uses a scheduler to distribute CPU time among runnable processes. On a four-core system, multiple processes can execute simultaneously, with the scheduler assigning available CPU cores according to scheduling policies and priorities.

When many computationally intensive programs run simultaneously, critical applications may experience delays. Process priority and CPU affinity can be used to improve responsiveness.

1. Check Running Processes:

The following command can be used to view CPU usage:

```
top
```

Alternatively:

```
ps -eo pid,ni,psr,%cpu,cmd
```

Here, NI represents the nice value and PSR shows the processor currently running the process.

2. Use Process Priorities:

A normal process can be started with a lower priority using:

```
nice -n 10 ./program
```

A critical process can be given a higher priority:

```
sudo nice -n -5 ./critical_program
```

An existing process can be adjusted using:

```
sudo renice -5 -p PID
```

Negative nice values generally give a process greater scheduling priority and require appropriate privileges.

3. Use CPU Affinity:

CPU affinity restricts a process to selected CPU cores.

For example, run a critical program on CPU core 0:

```
taskset -c 0 ./critical_program
```

A background workload can be restricted to other cores:

```
taskset -c 1-3 ./background_program
```

This prevents the background workload from competing directly with the critical process on the same core.

4. Proposed Four-Core Allocation:

A possible strategy is:

CPU Core Workload

Core 0	Critical application
Core 1	Critical/interactive application
Core 2	Background computation
Core 3	Background computation

This is only a starting point; actual allocation should be based on CPU utilization and workload characteristics.

5. Monitor the Result:

After applying affinity and priority changes, monitor CPU usage using:

```
top
```

and:

```
mpstat -P ALL 1
```

This helps determine whether CPU cores are properly balanced.

Conclusion:

Linux scheduling automatically distributes processes across available CPU cores. However, when critical applications require better responsiveness, nice and renice can adjust process priority, while taskset can control CPU affinity. Combining these techniques with monitoring can reduce CPU contention and improve responsiveness.

S.No. 9

Question:

A system administrator discovers that the /home directory is consuming almost all available disk space while /var has significant unused space. Analyze the problem and design a suitable storage-management strategy that prevents user files from affecting system services.

Answer:

The problem occurs because user files under /home are consuming most of the available disk space. If /home shares the same filesystem as /var and /, excessive user data can eventually prevent system services from writing logs, databases, temporary files, or other required data.

The solution is to separate user data from system data.

1. Analyze Disk Usage:

First, check filesystem usage:

```
df -h
```

Then determine which directories are consuming space:

```
sudo du -xh --max-depth=1 /home
```

Large files can be identified using:

```
find /home -type f -size +1G -ls
```

2. Create a Separate /home Partition:

A suitable partition can be allocated exclusively for /home.

For example:

/ → System files

/var → Logs and application data

/home → User files

swap → Virtual memory

This ensures that user files cannot consume space belonging to the root or /var filesystems.

3. Use the Available /var Space Carefully:

If /var has substantial unused capacity, the administrator can consider resizing the storage layout if the underlying partitioning scheme and filesystem support it.

For future deployments, using LVM is useful because logical volumes can be resized more flexibly.

For example:

Physical Volume

|

Volume Group

/ | \

LV LV LV

/ /var /home

This allows storage to be allocated according to actual usage.

4. Set User Quotas:

Disk quotas can prevent individual users from consuming excessive space.

For example, filesystem quotas can be configured so that each user has a maximum storage limit.

This prevents one user from filling the entire /home filesystem.

5. Clean Unnecessary Data:

Users should periodically remove unnecessary files, downloads, caches, and old data.

Large files can be located using:

```
du -ah /home | sort -h | tail
```

6. Monitor Disk Usage:

Regular monitoring can be performed using:

```
df -h
```

and:

```
du -sh /home/*
```

Alerts can be configured when disk utilization reaches a predefined threshold, such as 80% or 90%.

Conclusion:

The best long-term solution is to isolate /home from system files by using a separate partition or LVM logical volume. User quotas and regular monitoring should also be

implemented. This prevents user data from exhausting space required by /var, /, and critical system services.

S.No. 10

Question:

A Linux system becomes progressively slower after running continuously for several weeks. No hardware failure is reported. Analyze possible causes involving memory, processes, disk usage, and system resources, and propose a systematic troubleshooting procedure using appropriate Linux commands.

Answer:

A Linux system that becomes slower after running for several weeks may be experiencing memory pressure, excessive processes, disk-space problems, high I/O activity, or resource leaks. A systematic investigation should be performed instead of immediately restarting the system.

1. Check Overall System Load:

Start with:

`uptime`

This shows the system load average.

The `top` command provides an overview of CPU, memory, processes, and system load:

`top`

2. Check Memory Usage:

Use:

`free -h`

Check whether RAM is nearly exhausted and whether the system is heavily using swap.

Swap usage can be checked with:

`swapon --show`

For continuous monitoring:

`vmstat 1`

3. Identify CPU-Hungry Processes:

Use:

`ps aux --sort=-%cpu | head`

This identifies processes consuming the most CPU.

If a process is continuously consuming excessive CPU, investigate it before terminating it.

4. Check Memory-Hungry Processes:

Use:

```
ps aux --sort=-%mem | head
```

A process whose memory usage continually increases may indicate a memory leak.

5. Check Disk Space:

Use:

```
df -h
```

A filesystem that is almost full can cause applications and services to behave incorrectly.

Directory usage can be checked using:

```
sudo du -xh --max-depth=1 /
```

6. Check Inode Usage:

A filesystem can also run out of inodes even when free disk space remains.

Use:

```
df -i
```

If inode usage is very high, a large number of small files may be the cause.

7. Check Disk I/O:

High disk activity can make the system appear slow.

Commands such as:

```
iostat
```

and:

```
iotop
```

can help identify processes causing heavy disk I/O.

8. Check Running Services:

List active services using:

```
systemctl --type=service --state=running
```

Unnecessary services can be disabled if they are not required.

9. Check System Logs:

System logs can reveal errors, crashes, and repeated failures:

```
journalctl -p err
```

Recent kernel messages can be checked using:

```
dmesg | tail
```

10. Check Long-Running Processes:

Use:

```
ps -eo pid,etime,%cpu,%mem,cmd --sort=-%cpu | head
```

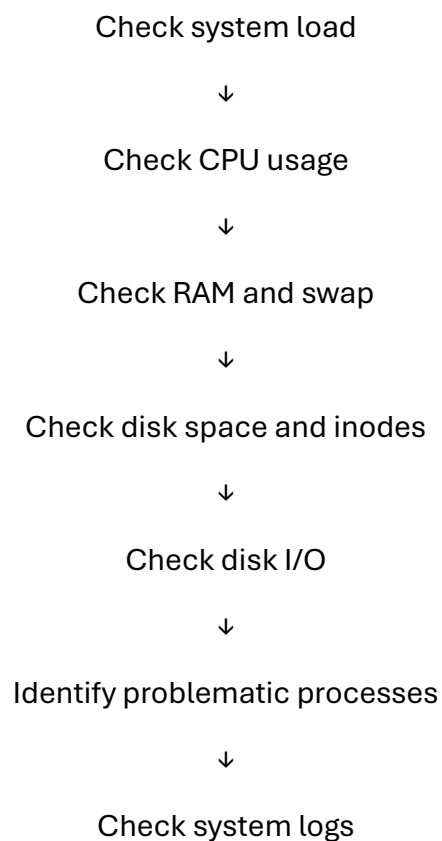
This helps identify processes that have been running for a long time while consuming significant resources.

11. Check for Resource Leaks:

If a process continuously consumes more memory, file descriptors, or CPU over time, it may have a resource leak. The process should be monitored and, if necessary, restarted individually rather than rebooting the entire server.

Systematic Troubleshooting Procedure:

The recommended sequence is:



↓

Optimize or restart only the affected service

↓

Monitor the system again

Conclusion:

Progressive system slowdown should be investigated systematically. Commands such as `top`, `free`, `vmstat`, `ps`, `df`, `du`, `iostat`, `iotop`, `systemctl`, and `journalctl` help identify the source of the problem. By correcting the specific resource issue and restarting only affected services when necessary, system stability can be restored without an unnecessary full-system reboot.